

INDUSTRIAL INTERNSHIP REPORT

on

“OmniClient – Multi-Client Service Marketplace Platform”

Prepared By:

Md Fahad

Internship Partner:

Upskill Campus

Project Repository:

GitHub Repository – OmniClient

Live Project:

OmniClient Live Deployment

EXECUTIVE SUMMARY

This report presents the complete work carried out during the Industrial Internship Program conducted by **Upskill Campus** in collaboration with UniConverge Technologies Pvt. Ltd. The internship focused on solving real-world industry problems using Full Stack Web Development technologies.

The assigned project, “OmniClient – Multi-Client Service Marketplace Platform”, is a full-stack MERN application developed to connect merchants and customers through a centralized digital marketplace. The platform enables merchants to register, create service listings, manage their businesses, and receive customer orders. Customers can browse services, add items to the cart, complete payments, place orders, and leave reviews and ratings.

The backend system was developed using Node.js, Express, and MongoDB following REST API architecture. Authentication and authorization were implemented using JWT tokens and role-based access control. Payment integration was implemented using Razorpay.

The frontend was designed using React, Tailwind CSS, and responsive UI principles to provide a professional user experience. The application was deployed successfully using Vercel.

This internship provided practical exposure to full-stack architecture, API development, authentication systems, database design, payment integration, responsive frontend development, and deployment workflows. It significantly improved technical problem-solving skills and industry-level development understanding.

TABLE OF CONTENTS	Page.No.
1.Preface	4
2.Introduction	6
2.1. About UniConverge Technologies Pvt. Ltd	
2.2. About Upskill Campus	
3. Objectives of Internship	9
4. Problem Statement	10
5. Existing and Proposed Solution	11
6. Proposed Design / System Architecture	13
7. High Level Design	15
8. Low Level Design	16
9. Technologies Used	17
10. Database Design	18
11.Backend/Frontend Implementation	22
12. Payment Integration	26
13. Admin Control System	27
14. Testing and Performance Evaluation	28
15. Challenges Faced	29
16. Learnings and Outcomes	30
17. Future Scope	31
18. Conclusion	32
19. References	33

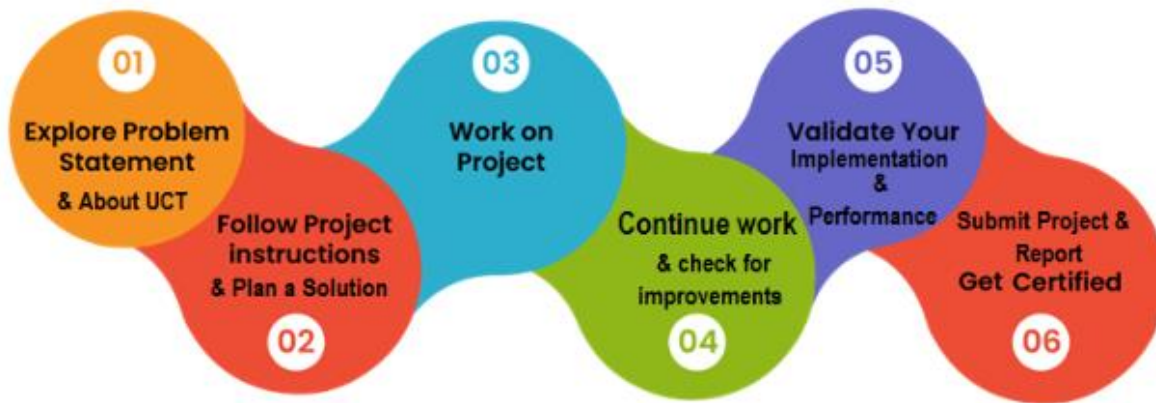
1. PREFACE

Industrial internships play an important role in bridging the gap between academic learning and practical industry implementation. This internship provided an opportunity to work on a real-world full-stack web application project using modern development technologies and industry workflows.

During this internship, the project “OmniClient – Multi-Client Service Marketplace Platform” was designed and developed from scratch using the MERN technology stack. The project involved backend API development, frontend integration, payment gateway implementation, database schema design, authentication systems, admin controls, and deployment processes.

The internship was structured week-wise with learning modules, implementation tasks, testing procedures, and final deployment. This helped in understanding professional software development practices, debugging workflows, and scalable project architecture.

The internship experience improved knowledge in frontend development, backend API structuring, MongoDB database management, authentication systems, responsive UI design, and cloud deployment. It also enhanced debugging abilities, logical thinking, and project management skills.



I would like to express my sincere gratitude to the mentors and organizers of Upskill Campus and UniConverge Technologies Pvt. Ltd. for providing valuable guidance and an excellent learning environment throughout the internship duration.

2. INTRODUCTION

The digital services industry is rapidly growing with increasing demand for online platforms that connect service providers and customers. Businesses require centralized systems where merchants can manage services while customers can discover and purchase services conveniently.

The OmniClient platform was developed to solve this requirement by creating a scalable multi-client marketplace system where merchants and customers interact through a secure web platform.

The project uses modern full-stack technologies and follows REST API architecture. The application supports merchant management, service listing, cart management, order processing, secure payments, reviews and ratings, and administrative controls.

The frontend application was developed using React and Tailwind CSS for responsive and professional UI development.

The backend APIs were developed using Node.js and Express with secure JWT authentication and MongoDB integration.

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end etc.



UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

* It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA

* It supports both cloud and on-premises deployments.

It has features to

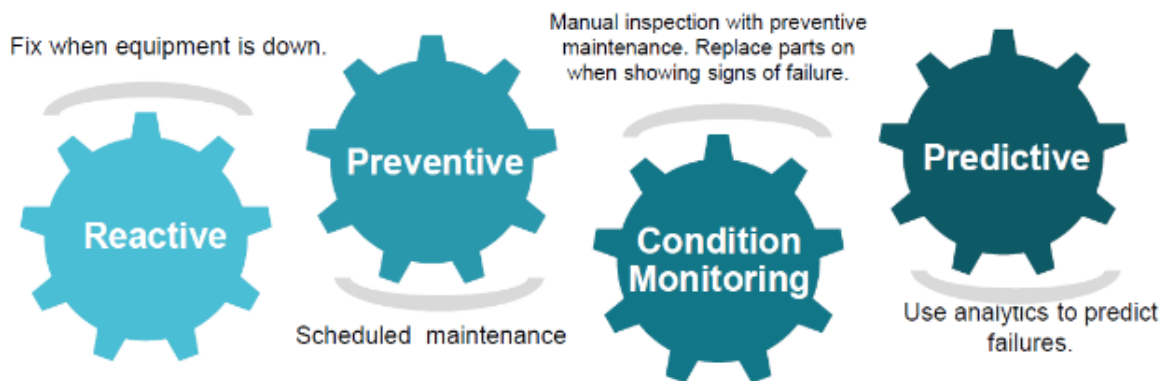
- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine

2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers personalized executive coaching in a more affordable, scalable and measurable way.

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.



3. OBJECTIVES OF INTERNSHIP

The major objectives of this internship were:

- To gain practical industry-level experience in Full Stack Web Development.
- To understand real-world software architecture and scalable application design.
- To develop a complete MERN stack application from scratch.
- To implement secure authentication and authorization systems.
- To learn REST API development and frontend-backend integration.
- To understand payment gateway integration using Razorpay.
- To gain experience in responsive UI development using React and Tailwind CSS.
- To improve debugging, testing, and deployment skills.
- To understand collaborative and modular development practices.

4. PROBLEM STATEMENT

Traditional service businesses often lack centralized digital platforms for managing customer interactions, service listings, payments, and orders. Customers also face difficulties in finding reliable service providers through a single platform.

The project objective was to build a scalable multi-client service marketplace where:

- Merchants can register and manage their services.
- Customers can browse and purchase services.
- Secure payments can be processed online.
- Reviews and ratings can improve trust.
- Admins can manage platform activities.

The platform needed to support multiple merchants and customers simultaneously while maintaining secure authentication and scalable architecture.

5. EXISTING AND PROPOSED SOLUTION

Existing Solutions

Existing marketplace systems often focus on a single category or provide limited customization for merchants. Many systems also lack integrated admin control, payment handling, or responsive interfaces.

Limitations include:

- Limited scalability
- Poor merchant management
- Complex UI/UX
- Lack of centralized dashboard
- Weak review and order systems

Proposed Solution

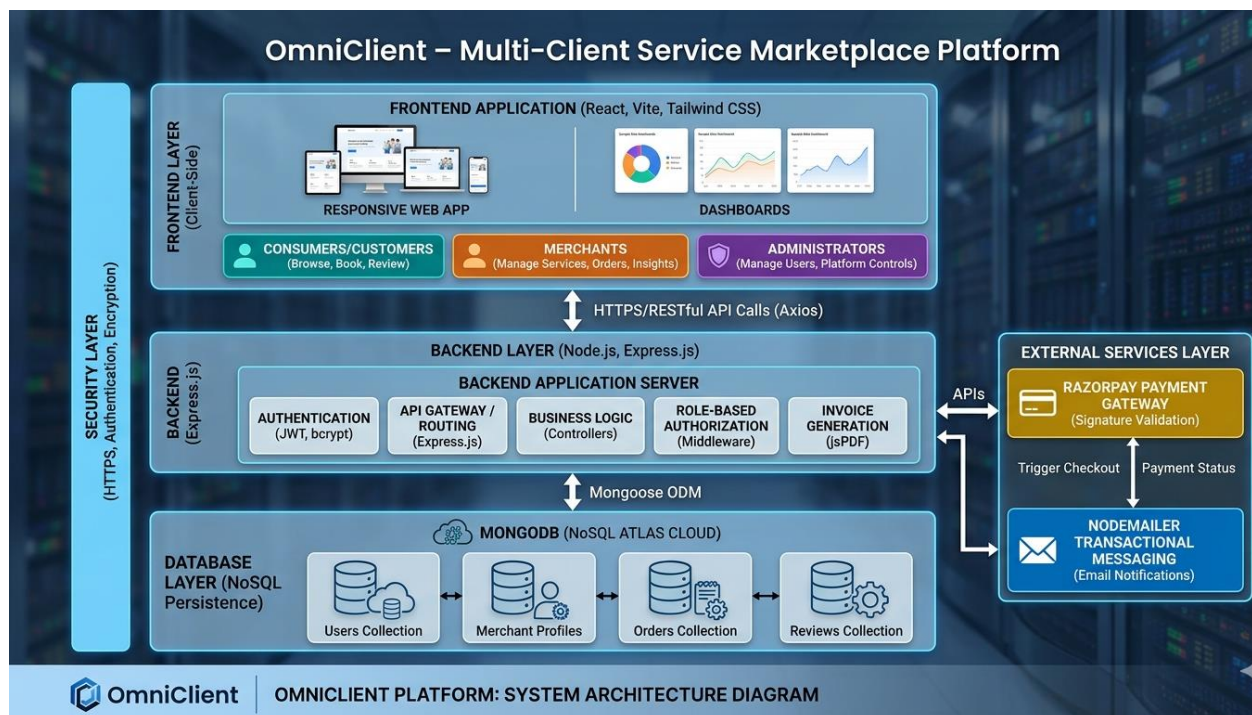
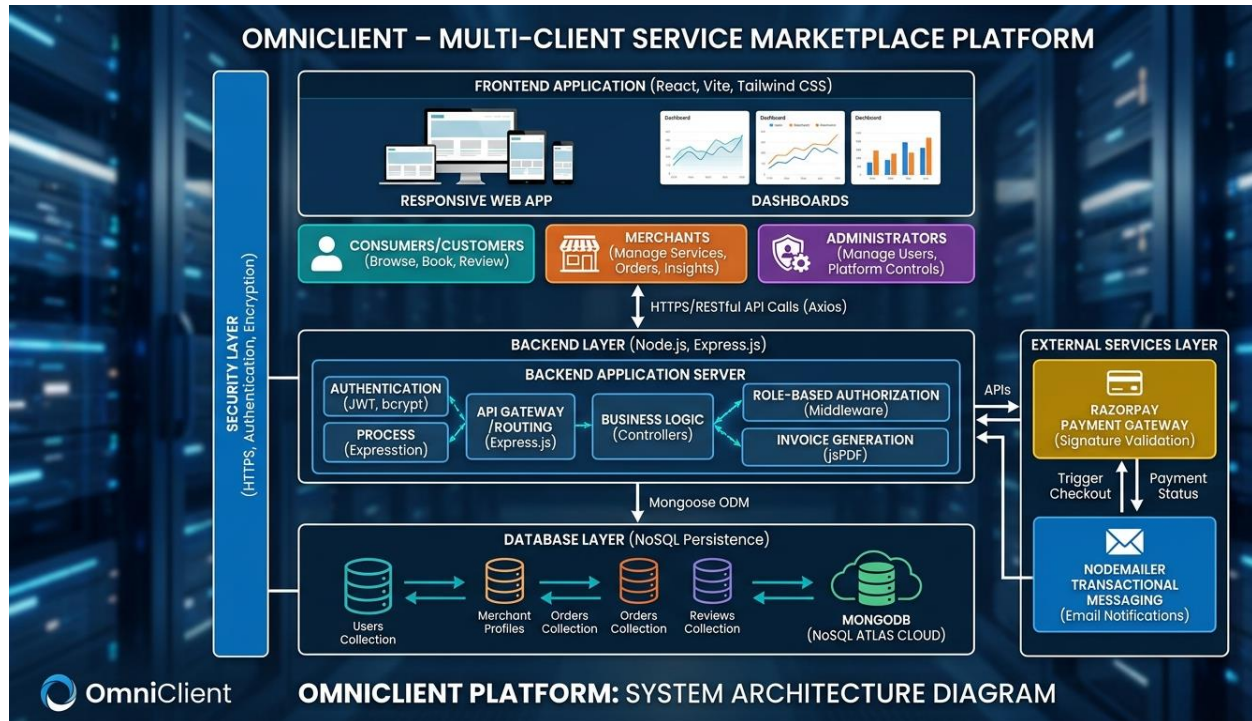
The proposed solution was the development of OmniClient, a modern MERN-stack digital marketplace platform that provides:

- Merchant onboarding system
- Customer service browsing
- Shopping cart functionality
- Secure order management
- Razorpay payment integration
- Reviews and ratings system
- Admin dashboard controls
- Responsive frontend UI

The platform follows modular architecture and supports scalable growth.

Existing Traditional Systems	Proposed System – OmniClient
Single service provider platforms	Multi-client scalable marketplace platform
Limited or manual merchant management	Dedicated merchant dashboard and management system
Restricted browsing experience	Easy browsing, searching, and ordering system
Basic login systems	Secure JWT-based authentication and authorization
Static or limited service listings	Dynamic service creation and management
No proper cart functionality	Integrated shopping cart and checkout system
Offline or limited payment methods	Secure online payment integration using Razorpay
Limited customer feedback features	Complete reviews and ratings system
Minimal admin control	Full admin dashboard with user and order management
Poor scalability handling	Scalable MongoDB architecture
Traditional and less responsive UI	Modern responsive UI using React and Tailwind CSS

6. PROPOSED DESIGN / SYSTEM ARCHITECTURE



7. HIGH LEVEL DESIGN

The high-level workflow of the application is:

1. User Registration/Login
2. Merchant Service Creation
3. Customer Browsing Services
4. Cart Management
5. Order Placement
6. Payment Verification
7. Review Submission
8. Admin Monitoring

The architecture uses role-based authentication and modular APIs to separate responsibilities across the application.

8. LOW LEVEL DESIGN

The low-level system implementation contains:

- Authentication APIs
- Merchant APIs
- Cart APIs
- Order APIs
- Review APIs
- Payment APIs
- Admin APIs

Database models were created for:

- Users
- Merchants
- Orders
- Reviews
- Cart

9. TECHNOLOGIES USED

Frontend Technologies

- React
- Tailwind CSS
- Axios
- React Router
- Vite

Backend Technologies

- Node.js
- Express
- JWT Authentication
- bcryptjs
- Nodemailer

Database

- MongoDB
- Mongoose ODM

Deployment

- Vercel
- MongoDB Atlas

10. DATABASE DESIGN

The database schema was designed using MongoDB collections.

User Collection

Stores:

- Name
- Email
- Password
- Role
- Authentication details

Merchant Collection

Stores:

- Business name
- Category
- Service list
- Merchant description

Order Collection

Stores:

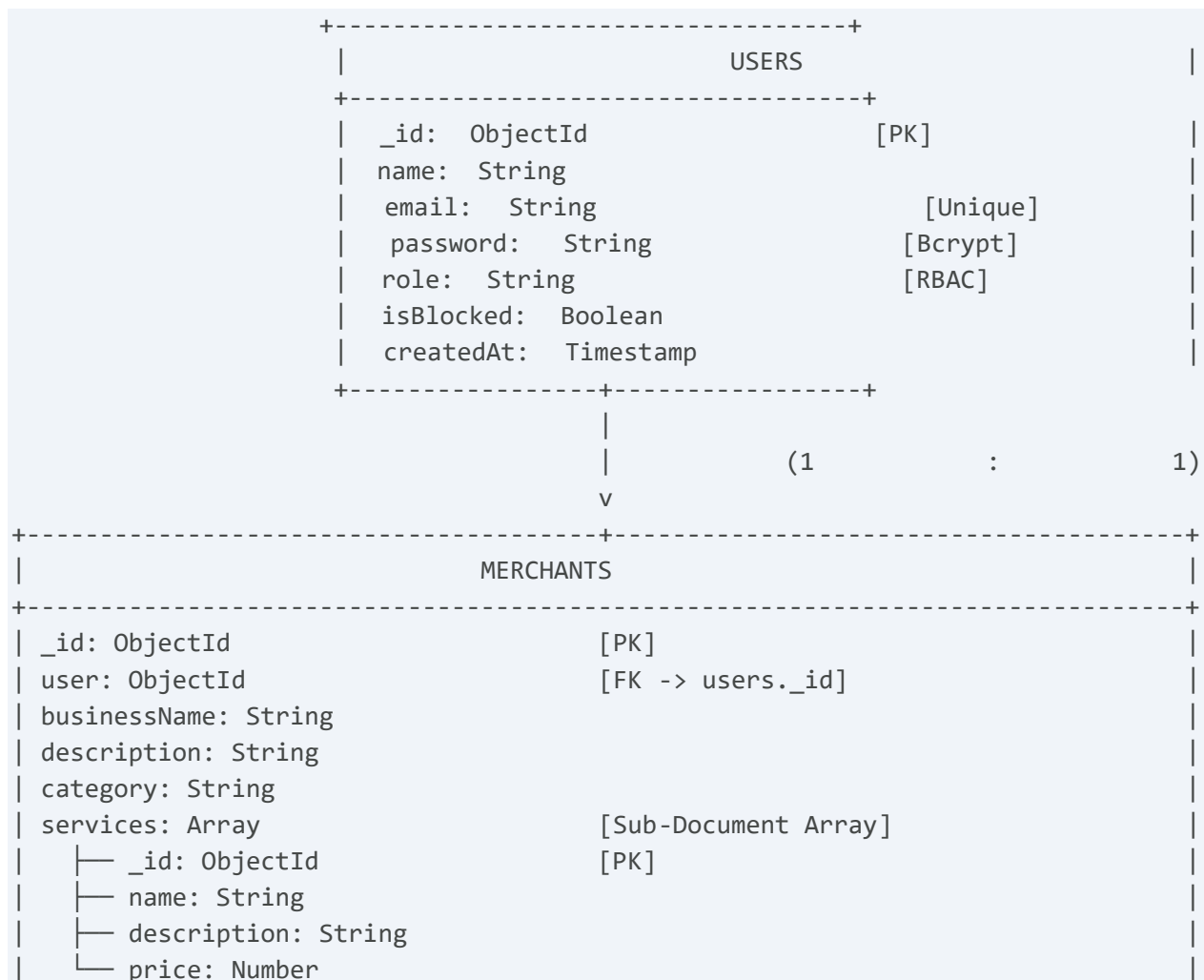
- Customer details
- Merchant details
- Payment information
- Order status

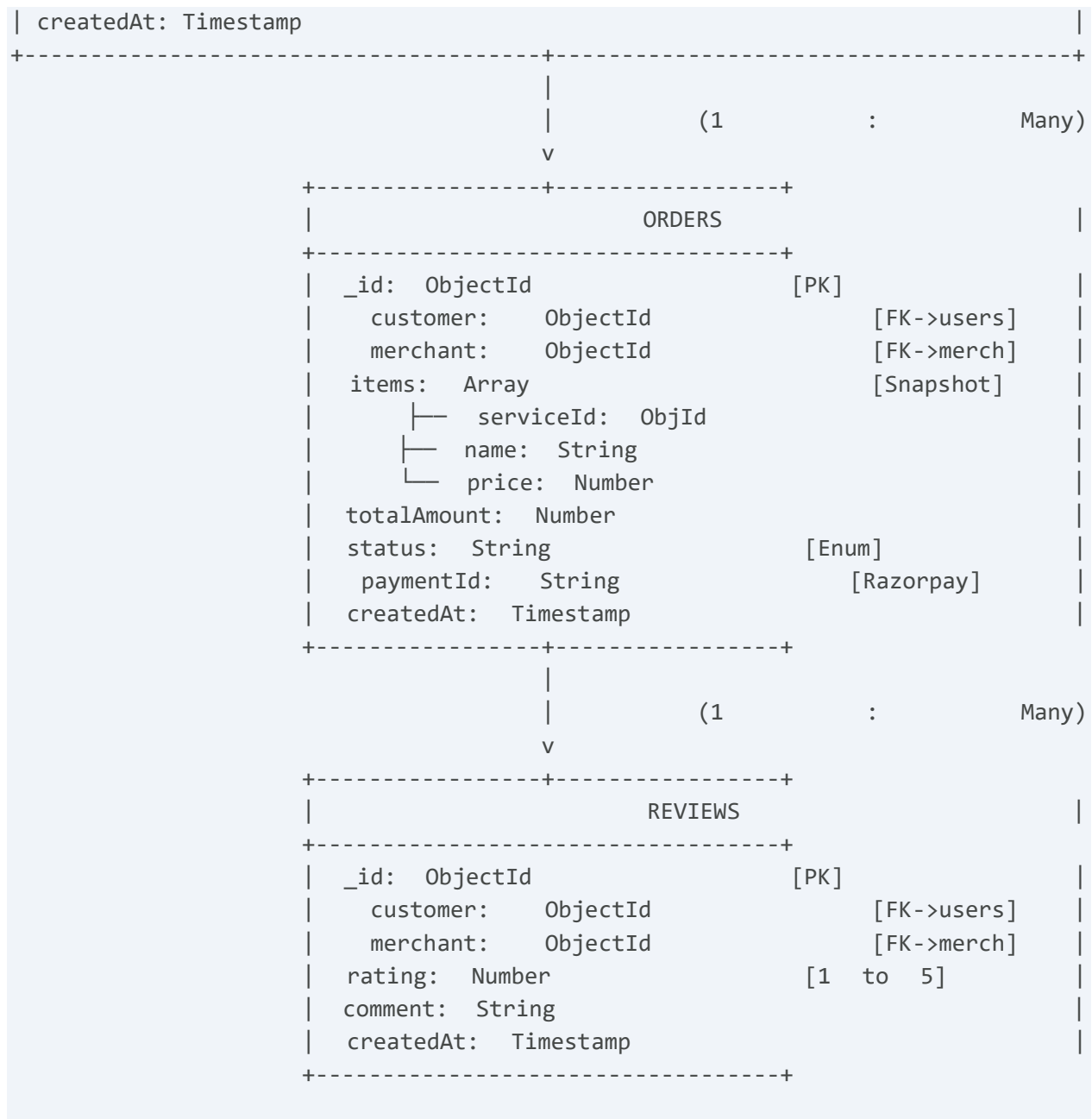
Review Collection

Stores:

- Ratings
- Customer comments
- Merchant reviews

Here is the visual representation of your database architecture, mapping out the collection schemas, field constraints, types, and document reference points inside MongoDB:





11. BACKEND IMPLEMENTATION

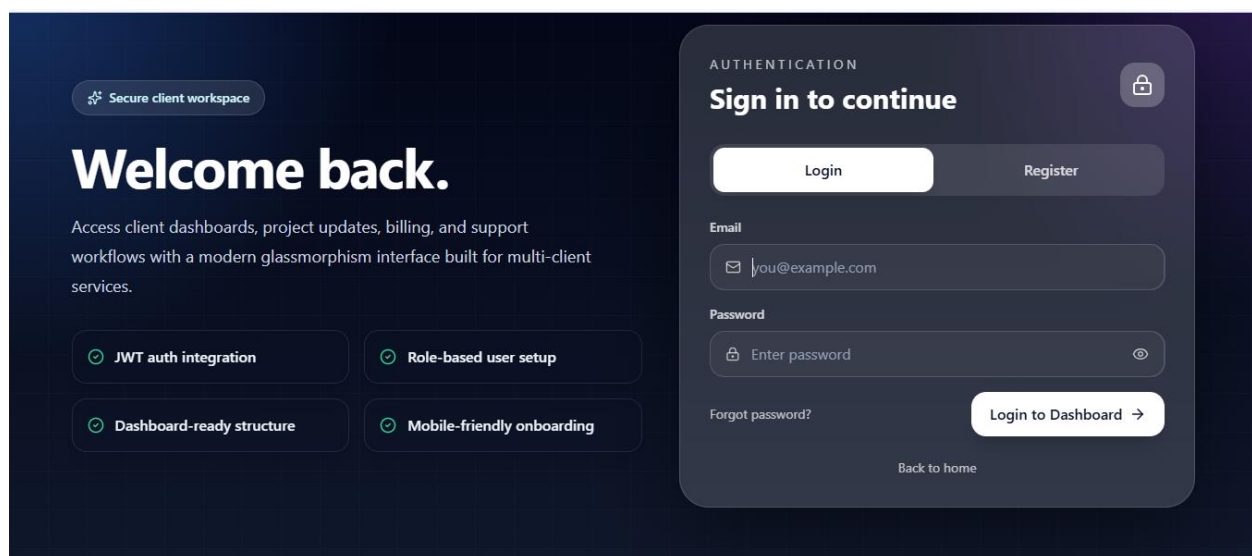
The backend was developed using RESTful architecture with modular route-controller-model structure.

Major backend modules included:

- Authentication Module
- Merchant Module
- Cart Module
- Order Module
- Payment Module
- Review Module
- Admin Module

Authentication was implemented using JWT tokens and middleware-based authorization.

The payment system integrated Razorpay order creation and payment verification APIs.



FRONTEND IMPLEMENTATION

The frontend UI was developed using React with responsive Tailwind CSS design principles.

Implemented pages include:

- Home Page
- Services Page
- Dashboard
- Client Portal
- Authentication Page
- Payment Success Page
- Order Details Page

The application uses reusable React components such as:

- Navbar
- Footer
- Dashboard Cards
- Service Cards
- Payment Components

[Home](#)
[Services](#)
[Dashboard](#)
[Client Portal](#)
[Login](#)
[Get Started](#)

Built for multi-client services

A sharper way to run client work.

A polished workspace for services, orders, billing, and support. Everything your team needs to manage many clients without losing speed or clarity.

Start Free Trial → Explore Services →

WORKSPACE SNAPSHOT

Multi-client control center

Live status

Revenue

\$47.9k +23%

This week

New clients	18
Open tasks	42
Resolved tickets	91%

[Home](#)
[Services](#)
[Dashboard](#)
[Client Portal](#)
[Login](#)
[Get Started](#)

WHY IT WORKS

Structured for service teams, built to feel premium.

The layout combines clarity, motion, and live-feeling panels so users immediately understand what the product does.

Secure Platform

Enterprise-grade security for client records, projects, and service access.

Fast Delivery

Keep teams aligned with clear task flow, updates, and checkpoints.

Multi-Client Control

Manage several customers, service plans, and order streams in one place.

Insights

Use live metrics to understand performance, load, and growth trends.

Our Services

Comprehensive solutions tailored to your business needs



Custom Development

Tailored solutions built to your exact specifications

Starting at \$500

Learn More



Cloud Solutions

Scalable cloud infrastructure and deployment

Starting at \$300/mo

Learn More



Security Services

Advanced security audits and protection

Starting at \$200/mo

Learn More



Data Analytics

Comprehensive data analysis and insights

Starting at \$400/mo

Learn More

<https://upskillcampus.vercel.app/services>

Ready to Transform Your Business?

Move from scattered tools to one clean client management experience that looks modern and feels fast.

Get Started Now →

View Client Portal

BUILT FOR MULTI-CLIENT TEAMS

Everything you need to run

12. PAYMENT INTEGRATION

Secure payment processing was implemented using Razorpay.

Features implemented:

- Payment order creation
- Razorpay checkout popup
- Signature verification
- Payment success handling
- Order confirmation

13. ADMIN CONTROL SYSTEM

An admin control system was implemented for platform management.

Features include:

- User management
- Merchant monitoring
- Order tracking
- User blocking/unblocking
- Role-based authorization

Admin routes were protected using middleware validation.

14. TESTING AND PERFORMANCE EVALUATION

The system was tested using:

- Postman API testing
- Browser testing
- Responsive UI testing
- Authentication testing
- Payment testing

Major test cases:

- User registration/login
- Merchant creation
- Cart operations
- Order placement
- Payment verification
- Review submission
- Admin access control

Performance observations:

- Fast API response times
- Responsive UI rendering
- Stable MongoDB integration
- Secure JWT authentication
- Successful payment verification

15. CHALLENGES FACED

Several challenges were encountered during development:

- Designing scalable MongoDB schemas
- Understanding API dependency flow
- Managing authentication tokens
- Debugging route conflicts
- Razorpay environment variable configuration
- Frontend-backend integration
- Responsive UI optimization
- Managing role-based authorization

These challenges improved debugging and analytical problem-solving skills significantly.

16. LEARNINGS AND OUTCOMES

Major learnings from the internship include:

- Practical understanding of MERN stack architecture
- REST API development principles
- JWT authentication and middleware usage
- MongoDB schema relationship handling
- Payment gateway integration
- Responsive frontend design
- Deployment workflows
- Real-world debugging techniques
- Role-based access control implementation

The internship provided strong exposure to industry-standard full-stack development practices.

17. FUTURE SCOPE

Future improvements that can be added include:

- Real-time chat system
- AI-based service recommendations
- Advanced analytics dashboard
- Multi-language support
- Push notifications
- Mobile application development
- Subscription plans for merchants
- Advanced search and filtering
- Real-time order tracking

18. CONCLUSION

The OmniClient project successfully achieved the objective of creating a scalable multi-client service marketplace platform using modern full-stack technologies.

The application demonstrated successful implementation of:

- Authentication systems
- Merchant management
- Shopping cart workflows
- Order processing
- Payment gateway integration
- Reviews and ratings
- Admin controls
- Responsive frontend UI

The internship experience greatly improved practical development knowledge, debugging skills, software architecture understanding, and confidence in building industry-level applications.

19. REFERENCES

1. **React Framework Documentation:-** Meta Open Source. "React – A JavaScript library for building user interfaces," official documentation, [Online]. Available: <https://react.dev/>. [Accessed: May 19, 2026].
2. **Node.js Runtime Environment:-** OpenJS Foundation. "Node.js v20.x Runtime Environment Documentation," [Online]. Available: <https://nodejs.org/docs/>. [Accessed: May 19, 2026].
3. **Express.js Web Application Framework:-** Express Project Contributors. "Express - Fast, unopinionated, minimalist web framework for Node.js," [Online]. Available: <https://expressjs.com/>. [Accessed: May 19, 2026].
4. **MongoDB Data Architecture:-** MongoDB, Inc. "The MongoDB Manual - Document-based NoSQL Database Platform," [Online]. Available: <https://www.mongodb.com/docs/manual/>. [Accessed: May 19, 2026].
5. **Mongoose Object Data Modeling (ODM):-** Automattic. "Mongoose ODM v8.x v8.x Documentation for MongoDB and Node.js," [Online]. Available: <https://mongoosejs.com/docs/>. [Accessed: May 19, 2026].
6. **Razorpay Payment Gateway API Integration:-** Razorpay Software Pvt. Ltd. "Razorpay Developer Documentation: Standard Checkout and Webhook Integration Matrix," [Online]. Available: <https://razorpay.com/docs/payments/>. [Accessed: May 19, 2026].
7. **Tailwind CSS Engine:-** Tailwind Labs Inc. "Tailwind CSS - A utility-first CSS framework for rapid UI development," [Online]. Available: <https://tailwindcss.com/docs/>. [Accessed: May 19, 2026].
8. **RESTful API Architectural Design Patterns:-** R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Department of Information and Computer Science, University of California, Irvine, 2000.
9. **JSON Web Token (JWT) Security Specifications:-** M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force (IETF), RFC 7519, May 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>.

10. **Full-Stack Application Patterns (MERN Stack)** E. Choi and V. Shrivastava, *Pro MERN Stack: Full Stack Web App Development with React, Express, MongoDB, and Node.js*, 2nd ed. New York, NY: Apress, 2019.